CS 5200 Project Proposal

Real-Estate Database Management System

Group Information

Group Name: SankaranVIdayachandiranG
Group Members: Vaibhav Sankaran, Gangatharan Idayachandiran

Top-Level Project Description

This project aims to create a comprehensive Real Estate Database Management System (DBMS) that facilitates property listings, client management, agent operations, and transaction tracking. The system will serve as a central platform for managing all aspects of real estate operations, from property listing to final sale or rental.

Primary Functionality

The application will provide functionality for three main user types:

1. Real Estate Agents:

- Create and manage property listings with detailed information (location, price, specifications)
- Schedule and track appointments with clients
- View their performance metrics (total sales, commission earned, ratings)
- Manage their client portfolio
- Complete transaction records when properties are sold or rented

2. Clients (Buyers/Renters):

- Search for available properties based on multiple criteria (price range, location, bedrooms, property type)
- Schedule viewing appointments for properties of interest
- Track their application/viewing history
- Submit reviews and ratings for properties and agents
- Update their preferences and budget information

3. System Administrators:

- Manage user accounts (agents and clients)
- Monitor system-wide statistics and analytics
- Oversee all transactions and ensure data integrity
- Generate reports on market trends and agent performance

Core Operations

The system will support all CRUD (Create, Read, Update, Delete) operations across multiple entities:

- **Create:** New property listings, user accounts, appointments, transactions, reviews
- **Read:** Search properties, view agent profiles, check appointment schedules, generate reports
- **Update:** Property status, appointment details, user information, transaction records
- **Delete:** Outdated listings, cancelled appointments, inactive user accounts

---

Data Domain Description

The Real Estate Database Management System manages comprehensive information about properties, users, transactions, and related real estate operations through a normalized relational database with 10 interconnected tables.

The Users table serves as the foundation for system authentication, storing user_id as the primary key, along with email, password_hash, first_name, last_name, phone, and user_type which categorizes users as admin, agent, or client. This table implements a generalization relationship where each user record can be specialized into either an agent or client profile through a one-to-one relationship.

The Agents table extends Users with real estate professional information, containing agent_id as primary key and user_id as a foreign key. Each agent profile is linked to exactly one user account, while one user can have at most one agent profile. Agent attributes include license_number, agency_name, commission_rate (stored as a decimal percentage), specialization, years_experience, rating (automatically updated through triggers), and total_sales. One agent can list zero to many properties and can facilitate zero to many transactions throughout their career.

The Clients table also extends Users, storing client_id as primary key and user_id as foreign key, maintaining a one-to-one relationship with the Users table. Client attributes include preferred_contact_method, budget_min, budget_max (representing their price range), preferred_location, and looking_for (indicating whether they want to buy, rent, or sell). One client can schedule zero to many appointments and can complete zero to many transactions.

The Properties table is the central entity, storing comprehensive property listing information with property_id as primary key. Each property belongs to exactly one location through the location_id foreign key, is categorized by exactly one property type through property_type_id, and is managed by exactly one agent through agent_id. Property attributes include title, description, price, listing_type (sale or rent), bedrooms, bathrooms, square_feet, lot_size, year_built, status (available, pending, sold, rented, or off_market), listed_date, sold_date, parking_spaces, and boolean fields for has_garage, has_pool, and has_garden. One property can have zero to many images, zero to many appointments for viewings, zero to many transactions (such as multiple rental periods), and zero to many reviews from different clients.

The Locations table stores physical addresses with location_id as primary key, containing street_address, city, state, zip_code, country, and optional latitude and longitude coordinates. One location can be associated with zero to many properties, reducing data redundancy by storing address information once and referencing it multiple times. The PropertyTypes table categorizes properties through property_type_id as primary key, with type_name and description attributes. Values include Single Family Home, Condo, Townhouse, Apartment, Commercial, Land, and Multi-Family. One property type can classify zero to many properties.

The PropertyImages table maintains multiple images per property with image_id as primary key and property_id as foreign key with cascade delete behavior. Each image record stores image_url, caption, is_primary (designating the main display image), and display_order for sequencing. One property can have zero to many images, demonstrating a one-to-many composition relationship where images are dependent on their parent property.

The Appointments table manages property viewing schedules through appointment_id as primary key, creating an associative relationship between properties, clients, and agents. It contains three foreign keys: property_id, client_id, and agent_id, all with cascade delete behavior. One property can have zero to many appointments scheduled over time, one client can book zero to many property viewings, and one agent can manage zero to many viewing appointments. Appointment attributes include appointment_date (datetime), duration_minutes, status (scheduled, completed, cancelled, or no_show), and notes for special requests.

The Transactions table records completed sales and rentals with transaction_id as primary key and three foreign keys: property_id, client_id, and agent_id, all with restrict delete behavior to preserve transaction history. One property can have zero to many transactions throughout its lifecycle, one client can complete zero to many property purchases or rentals, and one agent can facilitate zero to many deals. Transaction attributes include transaction_type (sale or rental), transaction_date,

final_price, commission_amount (automatically calculated from the agent's rate), payment_status (pending, completed, failed, or refunded), and optional lease_start_date and lease_end_date for rental agreements.

The Reviews table stores client feedback with review_id as primary key. It contains client_id as a foreign key with cascade delete, and optional property_id and agent_id foreign keys with set null delete behavior, allowing reviews to reference either a property, an agent, or both. One client can write zero to many reviews, one property can receive zero to many reviews from different clients, and one agent can receive zero to many reviews based on their service quality. Review attributes include rating (integer constrained between 1 and 5), review_text, review_date (timestamp), and is_verified (boolean) for moderation.

Normalization

All tables are normalized to **Third Normal Form (3NF)**:

Database Choice: SQL vs NoSQL

Selected Database: MySQL (Relational Database)

Rationale:

1. **Complex Relationships:** The real estate domain involves many interconnected entities with clear relationships (properties, agents, clients, transactions)
2. **Data Integrity:** ACID properties are crucial for financial transactions and preventing double-bookings
3. **Structured Data:** Property information is highly structured with consistent attributes
4. **Query Complexity:** Need for complex JOIN operations across multiple tables (e.g., finding all properties in a location with a specific agent and available appointments)
5. **Referential Integrity:** Foreign key constraints ensure data consistency across related tables

---

Technical Specifications: Software, Apps, Languages, and Libraries

Backend Database

- **DBMS:** MySQL 8.0+
- **Database Tool:** MySQL Workbench (for schema design and testing)

Application Development

- **Programming Language:** Python 3.9+
- **Web Framework:** Django 4.2+

Python Libraries

- **PyMySQL:** MySQL database connector
- **django-crispy-forms:** Enhanced form rendering
- **python-dotenv:** Environment variable management
- **Pillow:** Image handling for property photos

Frontend

- **HTML5/CSS3:** User interface templates
- **Bootstrap 5:** Responsive design framework
- **JavaScript:** Client-side interactivity and validation

Development Tools

- **Git:** Version control
- **VS Code:** IDE

README (to create and run the project)

Installation & Setup

Prerequisites

Before starting, ensure you have the following installed:
- Python 3.8 or higher
- MySQL 8.0 or higher
- pip (Python package manager)
- Virtual environment support

Step 1: Clone the Project from GitHub

Navigate to your desired directory
```
cd Desktop
```

Clone the repository
```
git clone https://github.com/svaibhav07codez/Real-Estate-Management-System-Project.git
```

Navigate into the project folder
```
cd Real-Estate-Management-System-Project
```

Alternative (if git is not installed):

1. Go to: https://github.com/svaibhav07codez/Real-Estate-Management-System-Project
2. Click the green "Code" button
3. Click "Download ZIP"
4. Extract the ZIP file to your Desktop
5. Navigate to the extracted folder in terminal

```
cd Desktop/Real-Estate-Management-System-Project
```

Step 2: Set Up Virtual Environment

Windows:
```
python -m venv venv
venv\Scripts\activate
```

macOS/Linux:
```
python3 -m venv venv
source venv/bin/activate
```

You should see `(venv)` at the start of your command prompt.

Step 3: Install Required Packages

```
pip install django==4.2
pip install pymysql
pip install python-dotenv
pip install pillow
pip install django-crispy-forms
```

```
pip install crispy-bootstrap5
pip install cryptography - (MacOS)
```

Step 4: Database Setup

4.1 Import Database Schema

Option A - Using MySQL Workbench:
1. Open MySQL Workbench
2. Connect to your MySQL server
3. File → Run SQL Script
4. Select `real_estate_schema.sql`
5. Click "Run"

Option B - Using Command Line:
```
mysql -u root -p < real_estate_schema.sql
# Enter your MySQL root password when prompted
```

Step 5: Configure Django Project

5.1 Create .env File

Create a file named `.env` in the project root:

```
DB_NAME=real_estate_db
DB_USER=root
DB_PASSWORD=your_mysql_password_here
DB_HOST=localhost
DB_PORT=3306
SECRET_KEY=django-insecure-change-this-key
DEBUG=True
```

Important: Replace `your_mysql_password_here` with your actual MySQL password.

5.2 Configure PyMySQL

Open `manage.py` and add these lines at the very top (if missing):

```
import pymysql
pymysql.install_as_MySQLdb()
```

Step 6: Run Migrations

```
python manage.py migrate
```

Step 7: Run the Server

```
python manage.py runserver
```

The application will be available at: `http://127.0.0.1:8000/`

Running the Application

Starting the Application

Every time you want to run the application:

1. **Navigate to project directory:**
   ```bash
   cd path/to/real_estate_project
   ```

2. **Activate virtual environment:**
   ```bash
   # Windows
   venv\Scripts\activate

   # macOS/Linux
   source venv/bin/activate
   ```

3. **Start Django server:**

   ```bash
   python manage.py migrate
   ```
   ```bash
   python manage.py runserver
   ```

4. **Access the application:**
   - Homepage: `http://127.0.0.1:8000/`
   - Admin Panel: `http://127.0.0.1:8000/admin/`
   - Analytics Dashboard: `http://127.0.0.1:8000/analytics/`

Stopping the Application

Press **Ctrl+C** in the terminal to stop the server.

Project Interest and Rationale

1. Vaibhav Sankaran

This project is personally interesting and professionally relevant for several reasons:

Personal Interest & Professional Experience

My interest in real estate technology stems from my experience working as a **Software Developer Intern at JLL (Jones Lang LaSalle)**, one of the world's leading commercial real estate services firms. During my internship, I gained firsthand exposure to the complexities of real estate operations, property management systems, and the critical role that well-designed databases play in managing vast amounts of property data, client relationships, and transactions.

Working at JLL sparked my fascination with how technology can streamline real estate operations, from property listings and lease management to client relationship tracking and financial reporting. I

witnessed the challenges that real estate professionals face in managing multiple properties, coordinating with clients, scheduling site visits, and tracking deal pipelines. This real-world experience motivated me to build a comprehensive Real Estate DBMS that addresses these operational needs.

Real estate is a tangible, real-world domain that everyone interacts with at some point in their lives. Managing property listings, tracking appointments, and analyzing market trends are practical problems that benefit from a well-designed database solution. The complexity of managing relationships between agents, clients, properties, and transactions makes this an intellectually stimulating project that directly connects to my professional interests.

Educational Value

1. **Complex Schema Design:** Working with 10+ interconnected tables provides excellent practice in database normalization and relationship modeling, mirroring the complexity I observed in enterprise real estate systems at JLL
2. **Real-World Application:** The project mimics actual real estate CRM systems used by agencies like Zillow, Redfin, and Realtor.com, as well as commercial real estate platforms similar to those used at JLL
3. **Business Logic:** Implementing stored procedures for transactions, triggers for automated updates, and functions for calculations demonstrates advanced database programming applicable to real estate tech solutions
4. **Full-Stack Development:** Building a complete CRUD application with Django provides valuable web development experience that complements my internship experience

Career Relevance

Understanding how to design and implement database systems for complex business domains is a crucial skill for software engineers and data professionals. Real estate tech is a rapidly growing industry, and my experience at JLL combined with this project creates a strong foundation for careers in:

- **Commercial Real Estate Software** (companies like JLL, CBRE, Cushman & Wakefield)
- **PropTech startups** (emerging technology companies disrupting real estate)
- **Real estate platforms** (Zillow, Redfin, CoStar, LoopNet)
- **Property management companies** requiring sophisticated database solutions
- **Real Estate Investment Trusts (REITs)** with data-intensive operations

This project allows me to apply theoretical database concepts to a domain I've worked in professionally, making it both personally meaningful and career-enhancing.

Technical Challenge

The project offers opportunities to implement:

- Multi-user role management (admin, agent, client)
- Complex business logic (commission calculations, appointment scheduling conflicts)
- Data analytics (market trends, agent performance metrics)
- Transaction management with ACID properties
- Trigger-based automation (auto-updating ratings, canceling appointments)

2. <u>Gangatharan Idayachandiran</u>

This project reflects my goal of gaining practical, industry-relevant experience while strengthening my foundation in database management. The real estate domain offers a realistic business context to design, implement, and manage a relational database with tangible applications.

Technical Learning Objectives

- Database Design & Normalization: Defining entities such as Properties, Agents, Clients, and Transactions using 3NF to ensure data integrity and eliminate redundancy.

- Relationship Modeling: Applying one-to-many and many-to-many relationships through primary and foreign keys.
- Query Development: Implementing complex queries with joins, subqueries, and filters to support property searches, reports, and analytics.
- Transactions & Constraints: Ensuring ACID compliance and applying constraints (PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL) to enforce data consistency.
- Optimization: Using indexing and query tuning for performance improvements.

CRUD Operations

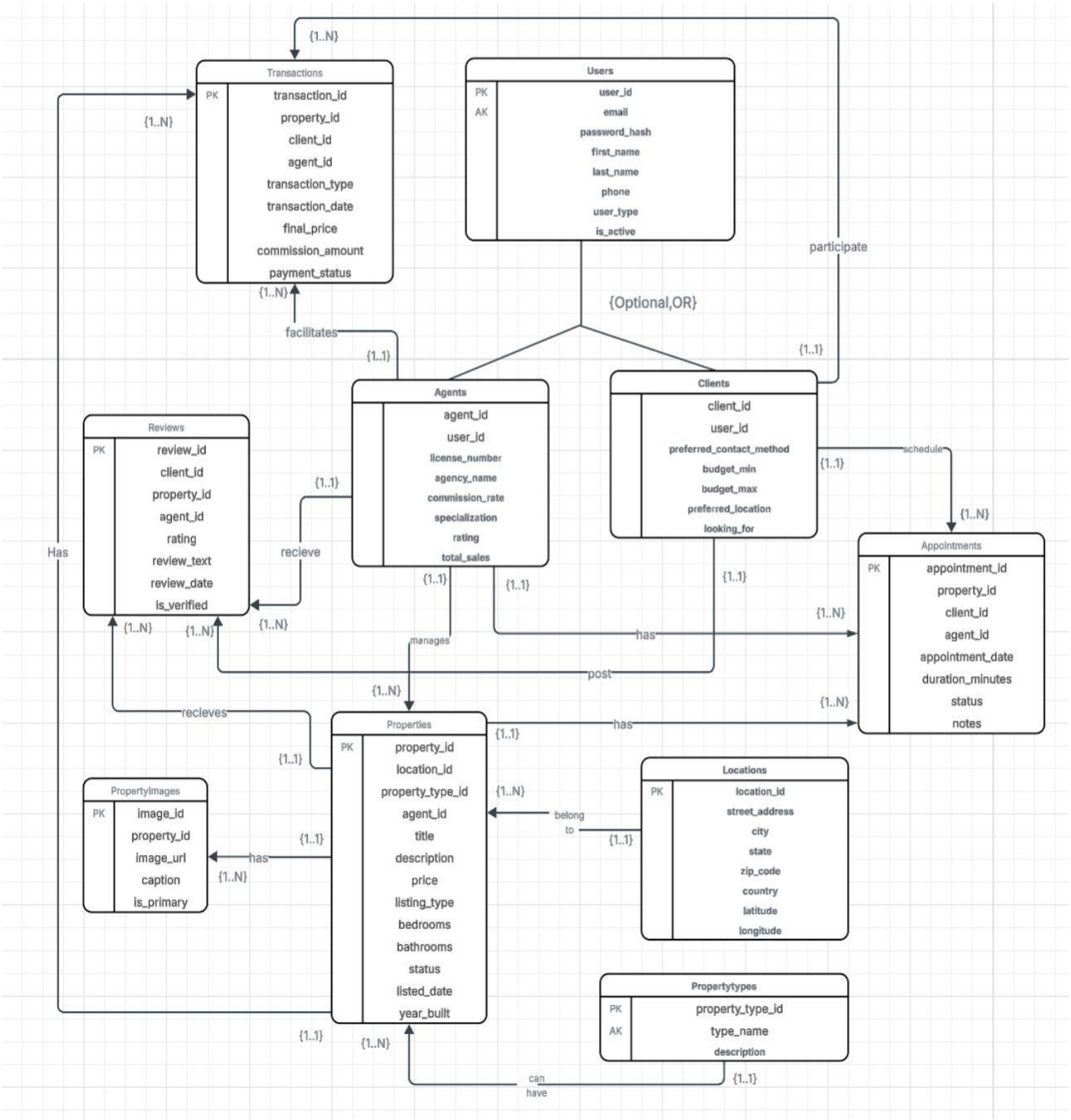
- Create: Add property listings, users, appointments, and transactions.
- Read: Retrieve listings, agent details, and performance reports.
- Update: Modify property status, user details, and appointment data.
- Delete: Remove outdated or invalid records while preserving integrity.

Career Relevance

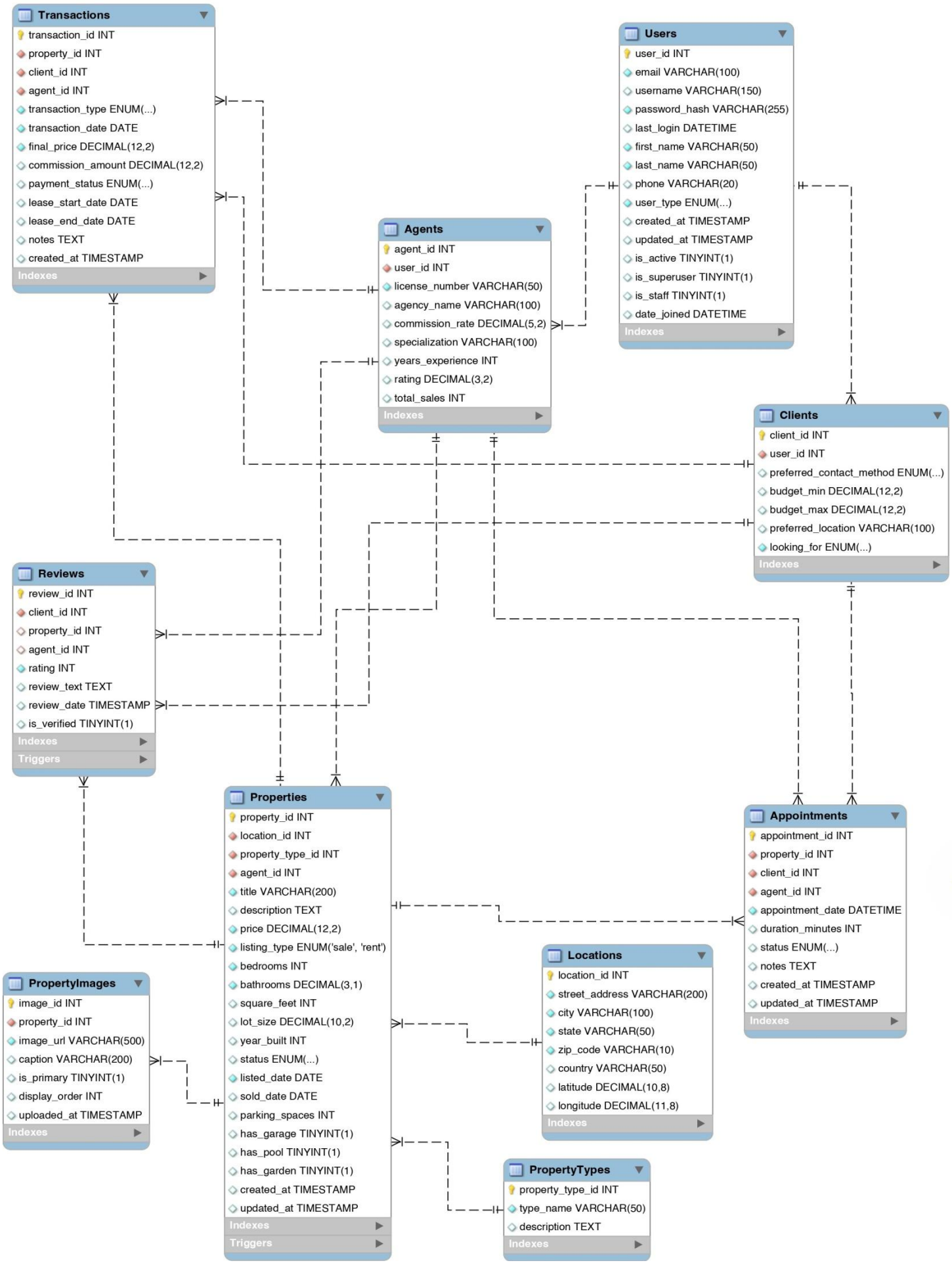
This project serves as a strong portfolio piece showcasing complete DBMS development—from schema design to query optimization. The skills gained are transferable across multiple domains such as e-commerce, finance, and healthcare, reinforcing both technical depth and practical problem-solving ability.

Conceptual Design: UML Diagram

Entity-Relationship Diagram



Reverse Engineer EER:



User Interaction Flow and Activity Diagram (Flow Chart)

For Clients (Buyers/Renters):

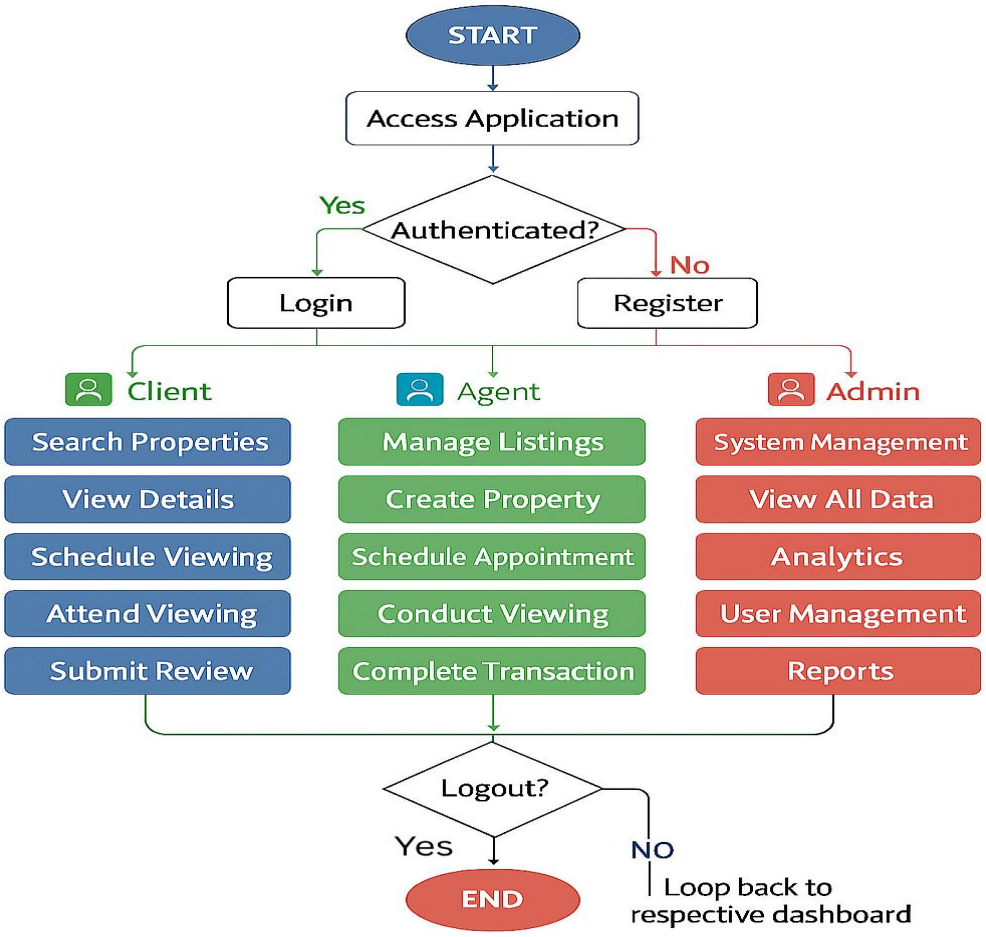
1. **Start Application**
   - Launch Django web application
   - Navigate to homepage
2. **User Authentication**
   - **Option A:** Register new account
     - Provide email, password, name, phone
     - Select user type: "Client"
     - Submit registration form
   - **Option B:** Login with existing credentials
3. **Set Preferences (First-time users)**
   - Enter budget range (min/max)
   - Select preferred location(s)
   - Choose looking for: Buy or Rent
4. **Search Properties**
   - **Search Options:**
     - Browse all available properties
     - Filter by price range
     - Filter by location (city/zip code)
     - Filter by property type (house, condo, apartment, etc.)
     - Filter by number of bedrooms/bathrooms
     - Filter by listing type (sale/rent)
   - View search results with property cards
5. **View Property Details**
   - Click on property from search results
   - See full property information:
     - Images gallery
     - Complete specifications
     - Location details
     - Agent information
     - Existing reviews
6. **Schedule Appointment**
   - Select "Schedule Viewing" button
   - Choose date and time
   - Add any special notes/requests
   - Submit appointment request
7. **Manage Appointments**
   - View "My Appointments" dashboard
   - See upcoming, completed, or cancelled appointments
   - Cancel or reschedule appointments if needed
8. **Submit Reviews**
   - After viewing, submit property review
   - Rate property (1-5 stars)
   - Write review text
   - Optionally review the agent
9. **Track Applications/Interest**
   - View list of properties viewed
   - Track appointment history
   - Monitor favorite properties
10. **Logout**
    - End session securely

For Agents:

1. **Start Application & Login**
    - Login with agent credentials
2. **Agent Dashboard**
    - View performance metrics:
        - Total active listings
        - Upcoming appointments
        - Recent transactions
        - Commission earned
        - Current rating
3. **Create New Property Listing**
    - Click "Add New Property"
    - **Step 1:** Enter property details
        - Title and description
        - Price and listing type (sale/rent)
        - Bedrooms, bathrooms, square feet
        - Year built, parking spaces
        - Features (garage, pool, garden)
    - **Step 2:** Enter location information
        - Street address, city, state, zip code
    - **Step 3:** Select property type
    - Submit new listing
4. **Manage Property Listings**
    - View all active listings
    - **Update Properties:**
        - Edit property details
        - Change price
        - Update status (available, pending, sold, rented)
        - Add/remove images
    - **Delete Properties:**
        - Remove outdated or cancelled listings
5. **Manage Appointments**
    - View appointment calendar
    - See scheduled viewings
    - Accept/decline appointment requests
    - Mark appointments as completed or no-show
    - Add notes after viewings
6. **Complete Transactions**
    - When property is sold/rented:
        - Navigate to property
        - Click "Complete Transaction"
        - Enter final sale/rental price
        - Add transaction notes
        - System automatically:
            - Calculates commission
            - Updates property status
            - Records transaction
            - Updates agent statistics
7. **View Client Information**
    - Access client profiles (privacy-compliant)
    - View appointment history with specific clients
    - Track client preferences
8. **Analytics & Reports**
    - View personal performance reports
    - Track commission earnings
    - Monitor property listing performance
    - View client reviews and ratings
9. **Logout**

For Administrators:

1. **Admin Login**
   - Access Django admin panel or custom admin interface
2. **User Management**
   - View all users (agents, clients, admins)
   - Create new user accounts
   - Edit user information
   - Deactivate/reactivate accounts
   - Delete users (with cascade handling)
3. **System Oversight**
   - Monitor all properties in system
   - View all appointments across agents
   - Review all transactions
   - Moderate reviews (verify/remove inappropriate content)
4. **Analytics Dashboard**
   - System-wide statistics:
     - Total properties listed
     - Properties sold/rented
     - Total transaction volume
     - Active users count
     - Average property prices by location
     - Top-performing agents
5. **Data Management**
   - Add new property types
   - Manage location data
   - Bulk data operations
   - Database maintenance
6. **Logout**



Database Programming Objects

To demonstrate mastery of MySQL and meet project requirements, the following database programming objects will be implemented:

Stored Procedures

1. **sp_create_property:** Creates a new property listing with transaction handling
2. **sp_schedule_appointment:** Schedules appointments with conflict checking
3. **sp_complete_transaction:** Processes property sales/rentals with automatic calculations

User-Defined Functions

1. **fn_price_per_sqft:** Calculates price per square foot for any property
2. **fn_agent_total_commission:** Returns total commission earned by an agent
3. **fn_avg_price_by_city:** Computes average property price for a specific city

Triggers

1. **tr_update_agent_rating:** Automatically updates agent rating when new reviews are added
2. **tr_check_property_delete:** Prevents deletion of properties with active appointments
3. **tr_cancel_appointments_on_status_change:** Auto-cancels appointments when property status changes to sold/rented

Views

1. **vw_available_properties:** Comprehensive view of all available properties with full details
2. **vw_agent_performance:** Agent performance metrics for reporting

---

Lessons Learned:

Technical Expertise Gained

MySQL Database Design and Normalization: Through this project, we gained hands-on experience designing a normalized relational database schema in Third Normal Form. We learned to eliminate partial and transitive dependencies while maintaining referential integrity through carefully defined primary keys, foreign keys, and constraints. Understanding ON DELETE and ON UPDATE actions was crucial for managing relationships between our 10 interconnected tables.

Stored Procedures, Functions, and Triggers: Implementing server-side database programming objects taught us how to encapsulate complex business logic directly in MySQL. We created stored procedures for multi-step operations like completing transactions, user-defined functions for calculations such as agent commission totals, and triggers that automatically update agent ratings and prevent invalid deletions. This demonstrated the power of event-driven database programming for maintaining data consistency.

PyMySQL and Raw SQL Implementation: Working directly with PyMySQL as our database driver provided deep insights into database connectivity. We learned to manage connections properly using try-finally blocks, execute parameterized queries to prevent SQL injection, manually handle transaction commits and rollbacks, and process result sets returned as dictionaries. Understanding cursor management and connection pooling was essential for building a reliable application.

Query Optimization: Building complex JOIN queries across multiple tables highlighted the importance of performance. We created indexes on frequently searched columns like city, price, and status, which significantly improved query response times. Learning to write

efficient WHERE clauses and understanding when different JOIN types are appropriate helped us optimize database operations.

Challenges and Insights

Connection Management: We initially encountered connection exhaustion issues when connections weren't being closed properly, teaching us to always ensure proper resource cleanup even when exceptions occur.

Time Management: Implementing raw SQL for all operations took longer than initially estimated compared to using an ORM, but provided much deeper understanding of database mechanics. The additional time invested in writing and testing SQL queries was valuable for learning.

Data Type Handling: We learned important lessons about type conversion between Python and MySQL, particularly handling decimal precision for financial data, datetime formatting for appointments, and boolean representation for property features.

Alternative Approaches: We considered using stored procedures for all CRUD operations but ultimately balanced this by using procedures for complex multi-step transactions while keeping simpler queries in Python for development flexibility.

Future Work:

Planned use:

The Real Estate Database Management System is designed to serve as a production-ready application for real estate operations. Following the successful completion of this academic project, we plan to deploy this system and share it with my manager and director at JLL (Jones Lang LaSalle), where I (Vaibhav S) worked as a Software Developer Intern. The application demonstrates practical database design principles that address real-world property management challenges I observed during my internship. Whereas, I (Gangatharan) plan to incorporate ML features to this application as mentioned below, and help build my profile.

Potential Areas for Added Functionality:

1. **Document Management**: The database could be expanded to store references to important real estate documents such as purchase agreements, inspection reports, and disclosure forms. New tables for Documents and DocumentTypes, linked to Properties and Transactions through foreign keys, would create a complete digital record system.

2. **Market Analytics and Price Predictions (ML)**: Additional database views and aggregate queries, along with a machine learning regression model that could be trained on historical transaction data to predict optimal listing prices for new properties, provide comparative market analysis, showing average prices by neighborhood, days on market statistics, and seasonal trends.
